

Day3 · 15차시

평가 & 좋은 테스트

‘되는 것 같다’ 를 ‘된다’ 로 바꾸는 법.
Day2에서 만든 에이전트를, 이제 믿을 수 있게 만든다.

SECTION 00

오프닝 — 만든 것을, 믿을 수 있게

코드를 만드는 것과, 그 코드를 믿는 것은 다르다

“데모는 멋졌는데 — 다음에도, 100번째도 똑같이 잘 될까?”

— 한 번 잘 된 건 운일 수 있다. 반복 가능한 증거가 평가다.

우리가 만든 에이전트를 다시 본다

▶ '만드는 것'과 '믿는 것'은 다르다 — 오늘은 믿을 근거를 만든다

Day2 — 만들었다

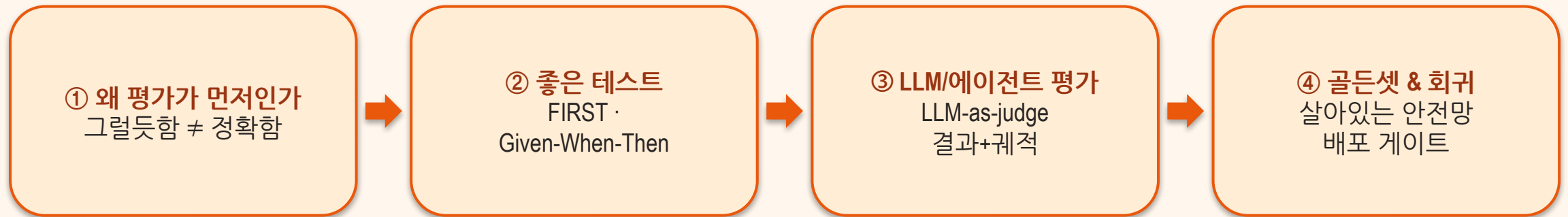
- 도구·메모리·RAG로 에이전트 완성
- 팀 MVP의 AI 기능이 돌아간다
- 데모 성공 — “되는 것 같다”

VS

Day3 — 믿는다

- 정말 잘 도는지 **증명하고 운영**
- 평가 → 운영 → 관찰 → 개선

이 차시 동안 얻는 것



SECTION 01

왜 평가가 먼저인가

그럴듯함 ≠ 정확함 — 유창함을 정확함으로 착각하지 말 것

유창한 오답 vs 검증된 정답

▶ 유창함은 정확함의 증거가 아니다

유창한 오답

- LLM은 항상 **자신감 있게** 답한다
- 문장은 매끄럽지만 내용은 틀림 — 환각
- 사람은 유창함을 정확함으로 착각
- “그럴듯하니 맞겠지”로 통과

VS

검증된 정답

- 기준(루브릭)으로 내용을 확인
- 근거·사실과 대조
- 표현이 달라도 의도를 충족
- **측정 가능한 증거**로 통과

평가 없는 개발의 비용

▶ 측정 없이는 개선도 없다 (building 9)

느낌 기반 테스트

질문 3개 던져보고 그럴듯하니 동작한다고 가정

조용한 회귀

고치면 다른 게 깨진다 — 그런데 아무도 모른다

증명 불가

누구도 “좋아졌다”를 증명하지 못한다

감으로 튜닝

비교 기준이 없으니 매번 처음부터

테스트되지 않은 에이전트는 신뢰할 수 없는 에이전트다.

무엇을 측정할지부터 정한다

▶ 막연히 '잘 되나' 말고 — 우리 MVP의 핵심 시나리오 하나를 정조준

히어로 시나리오

우선순위 높은 대표 사용 사례부터 겨눈다

정량 지표

정확도 · 응답시간 · 도구 정밀도/재현율

정성 지표

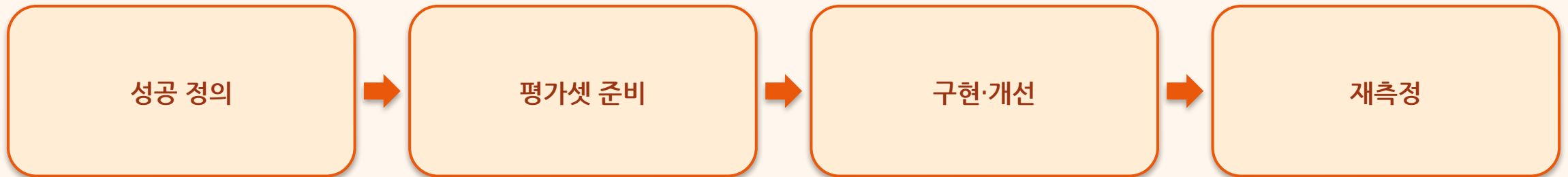
사용자 만족 · 응답 일관성

단일 진실원

핵심 지표의 source of truth 유지 → 회귀 조기 탐지

평가를 먼저 — 기능보다 측정 장치를

▶ 테스트는 사후 검사가 아니라 개발의 나침반 — '성공이 무엇인가'를 먼저 못 박는다



🔄 점수로 비교 → 나아질 때까지 반복

오프라인 평가 vs 온라인 평가

▶ 오프라인으로 회귀를 조기 감지하고, 온라인으로 현실 성능을 검증한다

오프라인 — 개발 중

- 평가셋에 대한 자동 테스트
- 빠르고 반복 가능·배포 전 차단
- 골든셋

VS

온라인 — 프로덕션

- 실제 사용자 상호작용 모니터링
- 현실 성능·엣지 케이스 발견

SECTION 02

좋은 테스트의 모양 — FIRST · Given-When-Then

행위 중심의 테스트

테스트가 잡아주는 것

▶ “돌려보고 눈으로 확인”을 자동·반복 가능하게 만드는 장치

약속의 고정

“이렇게 동작해야 한다”는
약속을 코드로 고정

즉시 알람

약속이 깨지면
즉시 알람

회귀 방지

한 번 잡은 버그를
두 번 안 만나게

안전벨트

리팩터링·기능 추가의
안전벨트

좋은 테스트의 5가지 성질

▶ 그리고 서로 얽힌 테스트는 결국 아무도 안 돌린다 → 죽은 테스트

F — Fast

느리면 아무도 자주 안 돌린다

I — Independent

서로 얽히지 않게 — 순서·상태 의존 금지

R — Repeatable

어디서·몇 번 돌려도 같은 결과 — LLM에선 가장 어렵다

S — Self-validating

통과/실패를 코드가 스스로 판정 (눈대중 X)

T — Timely

기능과 함께, 늦지 않게 작성

행위로 쓰는 테스트 — GWT

▶ 구현이 아니라 행위를 검증 — 한 문장씩, 비개발자도 읽힌다

Given 로그인한 사용자가 게시물을 1개 가지고 있을 때
When 그 게시물을 삭제하면
Then 목록에서 사라지고, 카운트가 0이 된다

코드로 옮기면

```
def test_delete_post():  
    user = login(); post = user.create_post() # Given  
    delete(post) # When  
    assert post not in user.posts # Then
```


평가 피라미드 — 단위에서 프로덕션까지

▶ 아래층이 빠른 피드백, 위층이 현실 검증 — 넷 다 필요하다

④ 프로덕션 모니터링

실시간 사용자 데이터 — 현실이 최종 시험대

③ 종단(E2E) 테스트

전체 워크플로 — 컴포넌트가 만나 생기는 **행동** 포착

② 통합 테스트

컴포넌트 상호작용 — 도구+계획+메모리가 함께 도는가

① 단위 테스트

도구·계획·메모리 각각 — 결함을 국소화

결정적 vs 비결정적

▶ 여기서부터 LLM 전용 평가가 필요해진다

일반 코드 (결정적)

- 같은 입력 → 항상 같은 출력
- `assert add(2,2) == 4`
- 정확 일치(==)로 검증 끝
- 실패하면 해석·디버깅 명확

VS

LLM (비결정적)

- 같은 입력 → 매번 다른 표현
- “요약이 핵심을 담았나?”
- 정답이 여럿 — == 로는 불통과
- → 새로운 평가 방법이 필요

SECTION 03

LLM/에이전트 평가 — 문자열 매칭으로는 안 된다

‘글자가 같은가’가 아니라 ‘기준을 충족했는가’

정확 일치(exact-match)의 함정

정확 일치 ✕

- “서울 인구는 약 940만” vs
- “약 9.4백만 명이 서울에 산다”
- → 둘 다 정답인데 `==`는 불통과
- 표현 다양성을 벌주면 좋은 답을 떨어뜨린다

VS

의미·기준 충족 ✓

- 뜻이 같으면 통과시켜야 한다
- 의미 유사도 (임베딩·BERTScore)
- 또는 기준 기반 채점
- BLEU/ROUGE 같은 n-gram도 정답을 낮게 평가

LLM 출력을 채점하는 3가지 방법

▶ 비용↓ 단순 → 비용↑ 유연 — 하나만 고르지 말고 섞어 쓴다

① 정확 일치 / 규칙

숫자·형식·금지어 — 결정적 체크에 적합 (싸고 빠름)

② 의미 유사도

임베딩 거리·BERTScore — 표현 달라도 뜻 비교

③ LLM 기반 (LLM-as-judge)

루브릭으로 채점 — 정확성·유창성·사실성 (G-Eval 등)

기준 기반 평가 — 무엇을 보는가

▶ 채점 기준(루브릭)은 사람이 먼저 정의한다 — '무엇이 좋은 답인가'의 명세

정확성 (Accuracy)

사실과 일치하는가
환각은 없는가

완전성 (Completeness)

물어본 것을
빠짐없이 답했는가

관련성 (Relevance)

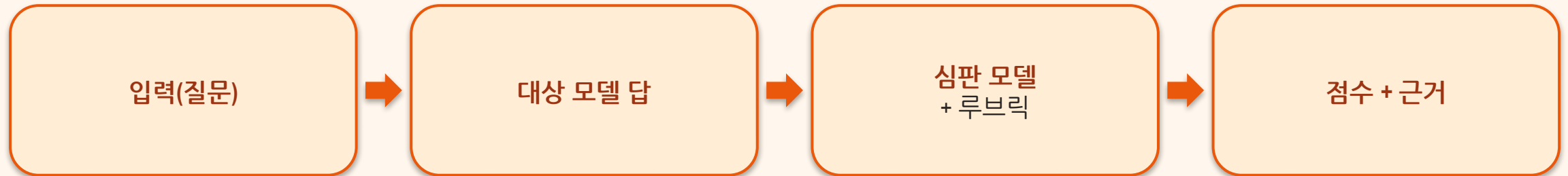
질문에 맞는 답인가
결길로 새지 않았나

형식 (Format)

요구한 형식·길이·톤을
지켰는가

채점을 다른 LLM에게 맡긴다

▶ 심판에게 루브릭 + 입력·출력을 주고 1~5점 + 이유를 받는다 → 대량 채점 자동화



LLM-as-judge, 맹신하지 말 것

▶ 심판도 LLM — 편향·일관성 한계 → 3중 안전장치

1

명확한 루브릭
모호하면 점수도 흔들린다

2

점수 + 근거 요구
왜 그 점수인지 검증

3

사람 라벨과 교차검증
정렬(agreement) 확인 — 사람 샘플 검토는 끝까지 유지

결과만이 아니라 ‘과정’도 본다

▶ 취소 에이전트 예: `cancel_order`를 정확한 주문ID로 불렀나? 답만 맞아도 과정이 틀리면 위험

도구 재현율 (Tool recall)

올바른 도구를 호출했는가

파라미터 정확도

인자를 정확히 채웠는가 — 전체가 아니라 19.99만 환불

호출 순서 (Trajectory)

올바른 순서로 단계를 밟았는가

최종 답 / 문구 재현율

결과 메시지에 필요한 표현이 담겼는가

결과 평가 vs 궤적 평가

▶ 답이 맞아도 — 운 좋은 추측·비효율·과잉 사고는 궤적에서만 보인다

결과 (Outcome)

- 최종 출력만 pass/fail
- 싸고 빠르다
- 운 좋은 추측을 못 거른다

VS

궤적 (Trajectory)

- 추론·도구 호출·인자·중간 출력 단계별 검사
- 비효율(불필요한 호출 남발)도 잡는다
- 비싸지만 원인 분석이 된다

SECTION 04

골든셋 & 회귀 방지 — 살아있는 안전망

평가의 단일 기준점을 만들고, 변경마다 다시 돌린다

골든셋이란 & 만드는 법

▶ 좋은 평가 예제 = 입력 상태 + 기대 결과

1

입력 수집

실제 사용자 요청·대표 시나리오를 모은다 (히어로 시나리오부터)

2

기대 결과/기준 라벨링

입력마다 '좋은 답'의 기준 또는 정답을 적는다

3

버전 고정

파일로 저장·버전 관리 — 평가의 단일 기준점

4

작게 시작

5~20개부터 — 한 케이스로 여러 행동을 동시에 검증

골든셋을 키우는 법 — 살아있는 명세

▶ 운영에서 터진 사고가 가장 값진 평가 케이스

실제 버그 추가
운영 사고를 케이스로
→ 같은 실수 두 번 안 함

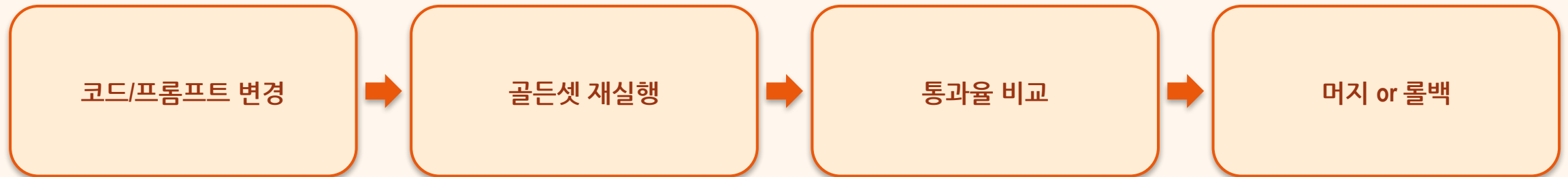
프로덕션 로그 마이닝
실데이터에서
대표·엣지 케이스를 캔다

모델 생성 + 사람 검수
모호성·적대적 케이스를
생성해 검수 후 편집

이렇게 자란 골든셋 = 에이전트가 할 일의 살아있는 명세(living spec)

변경할 때마다 다시 돌린다

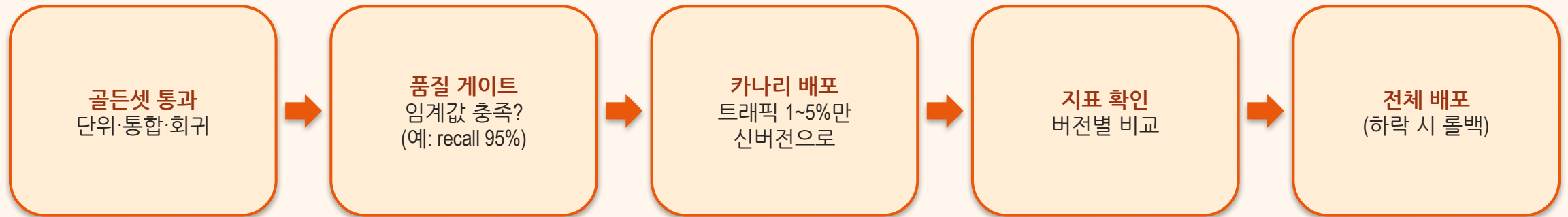
▶ CI에 걸면 품질 저하가 배포되기 전에 자동 차단



🔗 통과율 하락 → 머지 금지 → 고쳐서 재실행

품질 게이트 → 카나리 → 전체 배포

▶ 한 번에 전체가 아니라 — 단계마다 되돌아갈 다리를 남긴다



SECTION 05

정리

믿을 수 있는 평가란 무엇인가

오늘 3줄

- 그럴듯함 ≠ 정확함 — 기준(루브릭)으로 측정한다
- 좋은 테스트는 FIRST·GWT — LLM은 **LLM-as-judge**, 에이전트는 **결과+궤적**
- **골든셋 + 회귀 루프 + 배포 게이트**로 ‘좋아졌다/나빠졌다’를 증명한다

신뢰는 코드가 아니라 증거에서 나온다.

— 측정 없이는 개선도 없다